2️⃣

Expose Graphs on a Local Server

Overview

1. **Run a local server to expose our graph as an API**
2. **Run the exposed graph**

- Now we can go and run it by:
    1. Using Python notebooks with the following commands:
        - `client.runs.create`
        - `client.runs.stream`
    2. Using a front end
        - LangSmith Studio UI is pre-built for this
        - Or make your own UI

1. Run Local Server to Expose Graph

1. Create this directory:
    - `.env`

      ```
      # This must contain a LANGSMITH_API_KEY if we want to connect to LangSmith Studio UI
      LANGSMITH_API_KEY=your-key
      LANGSMITH_PROJECT=my-local-graph
      LANGSMITH_TRACING=true
      ```

    - `agent.py`
        - This should define, build & compile the graph
        - If we have separate files containing our sub-graphs we should have them here too
    - `langgraph.json`

      ```
      {
        "dependencies": ["."],
        "graphs": {
          "agent": "./agent.py:agent"
        },
        "env": ".env"
      }
      ```

2. Ensure dependencies installed in our environment
3. Navigate to this directory in terminal
4. Run `langgraph dev`
    - This spins up a local server on `http://localhost:2024` which exposes our agent

2. Run the Exposed Graph

Option 1: Call Via Python LangGraph SDK

We can use a .ipynb to call our exposed graph

```
from langgraph_sdk import get_sync_client

client = get_sync_client(url="http://localhost:2024")

for chunk in client.runs.stream(
    None,  # Threadless run
    "agent", # Name of assistant. Defined in langgraph.json.
    input={
        "messages": [{
            "role": "human",
            "content": "What is LangGraph?",
        }],
    },
    stream_mode="messages-tuple",
):
    print(f"Receiving new event of type: {chunk.event}...")
    print(chunk.data)
    print("\n\n")
```

▼ **Async Version**

```
from langgraph_sdk import get_client
import asyncio

client = get_client(url="http://localhost:2024")

async def main():
    async for chunk in client.runs.stream(
        None,  # Threadless run
        "agent", # Name of assistant. Defined in langgraph.json.
        input={
```

```
        "messages": [{
            "role": "human",
            "content": "What is LangGraph?",
            }],
        },
    ):
        print(f"Receiving new event of type: {chunk.event}...")
        print(chunk.data)
        print("\n\n")

asyncio.run(main())
```

Option 2: Go to `http://localhost:2024` = LangSmith Studio UI

- Now we can use chat interface, see interactive traces and graphs etc

Option 3: Go to LangSmith Agent Chat UI

1. Go to https://agentchat.vercel.app/
2. Input `http://localhost:2024` as your deployent URL, and your agent name and LangSmith API key
3. Now we can use our agent thrugh the chat interface and let users try it